

Done. I first checked the capabilities against the **current Semantica documentation**, then regenerated the project rather than simply adding more code to the previous version.

The current Semantica documentation confirms support for the capabilities used here: Knowledge Graph construction, provenance, ontology/SHACL validation, persistent graph stores, GraphRAG/context, change management, export and visualization.

 Semantica +3

Download the regenerated project

Download ABC + DeepWiki + Semantica POC v2 — Complete Project

I also ran a Python syntax check on all project Python files and verified the ZIP integrity — both passed.

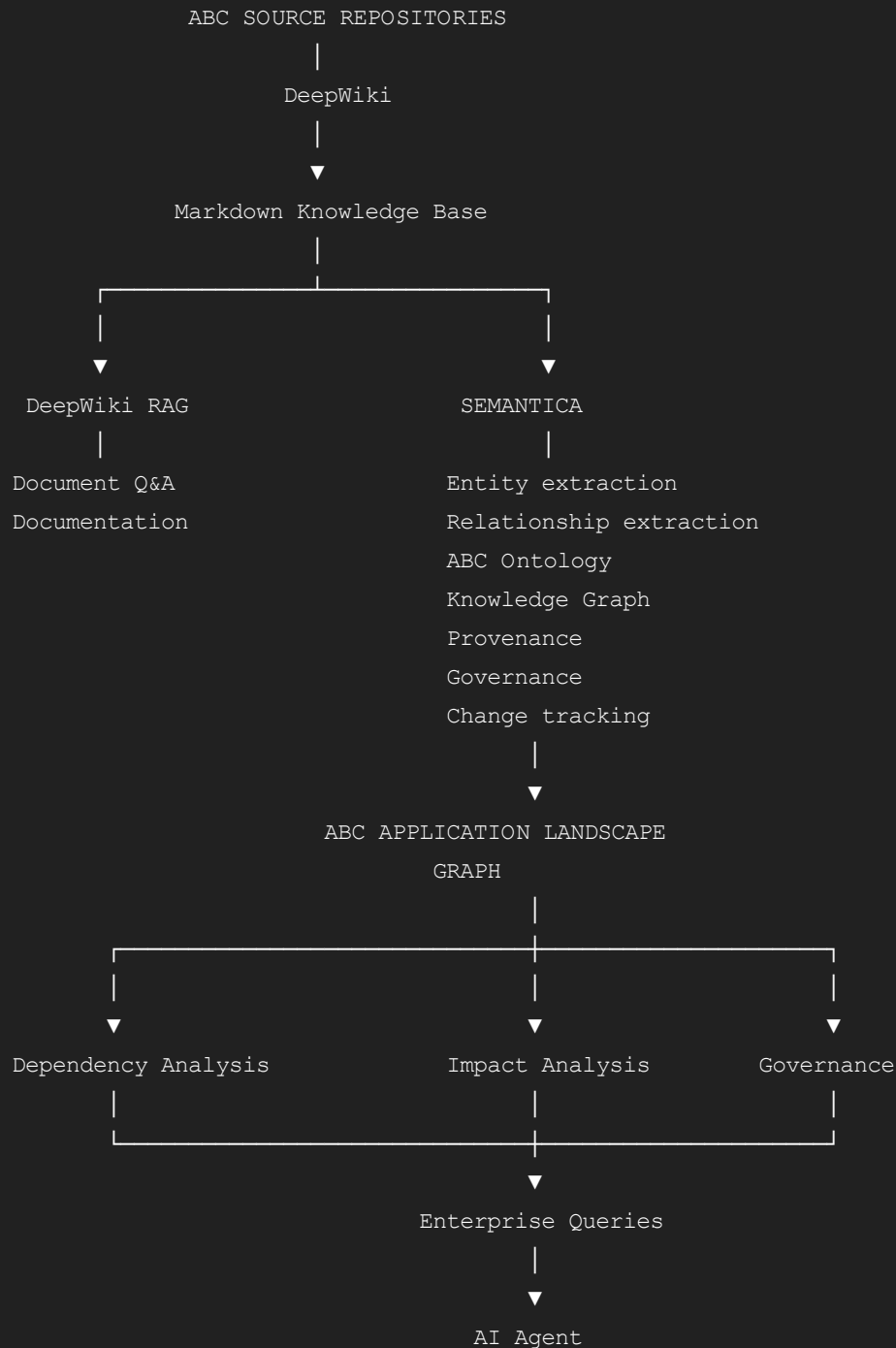
What was missing from the previous project

Capability	Previous project	V2
DeepWiki Markdown ingestion		
Application Landscape Graph	 basic	 expanded
Cross-application dependencies		
Reverse dependencies		
Multi-hop blast-radius analysis		
Kafka producer/consumer analysis		
Technology analysis		
Business capability analysis		
Owner/environment/database/API queries		

Capability	Previous project		V2
Enterprise landscape summary		✗	✓
Source provenance	Basic metadata		Semantica ProvenanceManager + ledger
ABC ontology		✗	✓
SHACL/ontology governance		✗	✓ Semantica OntologyEngine attempt + deterministic rules
Architecture governance report		✗	✓
Architecture change detection		✗	✓
Graph snapshots		✗	✓
Neo4j persistent graph option		✗	✓
RDF/Turtle/JSON-LD export		Partial	✓
Interactive graph visualization		Partial	✓
Optional Semantica LLM extraction		✗	✓
Production architecture documentation		✗	✓
Semantica usage instructions		Basic	✓ detailed

The most important change

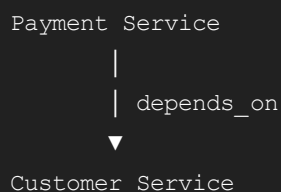
The new project explicitly implements this:



That is the actual purpose of Semantica in this project.

New important capability: provenance

For example:



The graph also retains:

```
source_file:
deepwiki_docs/payment-service/architecture.md

source_section:
Dependencies

confidence:
1.0

extraction_method:
abc_markdown_rules
```

Semantica's current architecture documentation states that provenance is built into graph construction and that nodes/edges can retain source lineage. [Semantica...](#)

This is extremely important for your eventual ABC implementation because the graph shouldn't simply say:

Payment Service depends on Customer Service.

It should be able to say:

This relationship came from this DeepWiki document and this section.

New: ABC architecture governance

The project now checks things such as:

```
Production Application
|
|— Owner?                REQUIRED
|
|— Business Capability?   REQUIRED
|
|— Environment?          REQUIRED
```

Run:

Bash

```
python governance.py
```

It produces:

```
output/governance_report.json
```

The project also attempts to use Semantica's `OntologyEngine`.

Semantica currently documents ontology generation, SHACL shape generation, graph validation, OWL/RDF export and ontology quality capabilities. [Semant...](#)

New: architecture change analysis

You can now take a snapshot:

Bash

```
python snapshot.py --output snapshots/baseline.json
```

Then update your DeepWiki Markdown.

Run:

Bash

```
python run_pipeline.py
```

Then:

Bash

```
python compare_snapshots.py \  
    snapshots/baseline.json \  
    output/abc_graph.json
```

You'll get:

```
Relationships Added  
Relationships Removed  
Nodes Added  
Nodes Removed
```

This is the beginning of **Architecture Drift Detection**.

New: persistent enterprise graph

For a small POC:

```
DeepWiki
  ↓
Semantica
  ↓
In-memory graph
```

is fine.

For ABC with hundreds/thousands of applications, the project now has an optional persistent graph path:

```
DeepWiki
  ↓
Semantica
  ↓
Neo4j / FalkorDB
  ↓
Persistent Application Landscape
```

The current Semantica documentation explicitly documents persistent graph storage including Neo4j and FalkorDB. [Semant...](#)

Example:

Bash

```
python run_pipeline.py \
  --neo4j-uri bolt://localhost:7687 \
  --neo4j-user neo4j \
  --neo4j-password password
```

New enterprise queries

You can now ask:

Bash

```
python query_cli.py summary
```

or:

Bash

```
python query_cli.py deps "Payment Service"
```

Bash

```
python query_cli.py reverse-deps "Customer Service"
```

Bash

```
python query_cli.py event-consumers "customer.updated"
```

Bash

```
python query_cli.py event-producers "payment.created"
```

Bash

```
python query_cli.py technology "Java 11"
```

Bash

```
python query_cli.py capability "Payments"
```

Bash

```
python query_cli.py owner "Payments Engineering Team"
```

Bash

```
python query_cli.py environment "Production"
```

Bash

```
python query_cli.py database "CustomerDB"
```

Bash

```
python query_cli.py blast "Customer Service"
```

Bash

```
python query_cli.py business-impact "Customer Service"
```

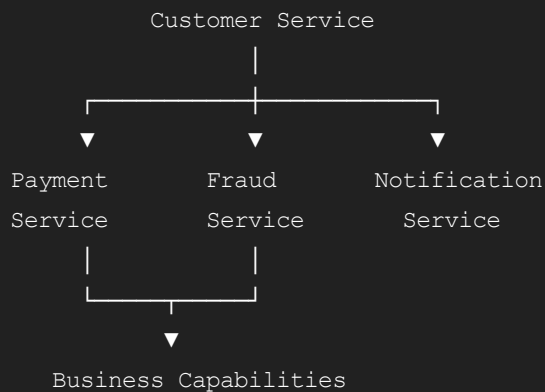
The most important query

For your ABC use case, I'd focus heavily on:

Bash

```
python query_cli.py blast "Customer Service"
```

Conceptually:



This is where Semantica starts providing something materially different from simply searching DeepWiki Markdown.

I also added a very important production guideline

The project **does not blindly send all Markdown to an LLM and ask it to invent a graph**.

The default pipeline is:

```
DeepWiki Markdown
  ↓
Controlled ABC extraction
  ↓
Semantica Graph
```

This is deliberate.

For architecture data, you don't want:

LLM thinks Payment Service probably depends on Customer Service.

You want:

DeepWiki document explicitly states the dependency → extract it → put it into graph → preserve evidence.

For less-structured DeepWiki documents, the project includes:

```
optional_llm_extraction.py
```

which demonstrates Semantica's semantic extraction APIs. The current Semantica documentation supports both pattern-based extraction and LLM-based extraction.

← Semant...

Project documentation included

The ZIP now contains:

```
docs/SEMANTICA_USAGE.md
```

Explains:

- installation
- where Semantica sits
- how to process DeepWiki
- graph construction
- provenance
- ontology
- SHACL
- persistent graph
- LLM extraction
- change analysis
- production recommendations

```
docs/ABC_ONTOLOGY.md
```

Defines the initial ABC ontology:

```
Application
Service
API
KafkaTopic
Database
DataEntity
Technology
Framework
BusinessCapability
BusinessDomain
```

```
Team
Owner
Environment
ExternalSystem
Document
```

and relationships such as:

```
depends_on
exposes_api
consumes_api
publishes_event
consumes_event
uses_database
uses_technology
supports_capability
owned_by
deployed_in
```

```
docs/PRODUCTION_ARCHITECTURE.md
```

Shows how I would evolve this from the POC into the actual ABC enterprise architecture.

One important note about accuracy

I have **not claimed that I executed Semantica itself locally**, because this execution environment does not have the Semantica package installed. I verified the implementation against the current official Semantica documentation, and the project includes graceful handling for optional Semantica modules whose APIs can vary by installed release. The current official docs show the package/API capabilities used here. [Semant... +1](#)

The Python code itself has been syntax-checked successfully.

Download: [ABC Semantica DeepWiki POC v2](#) 